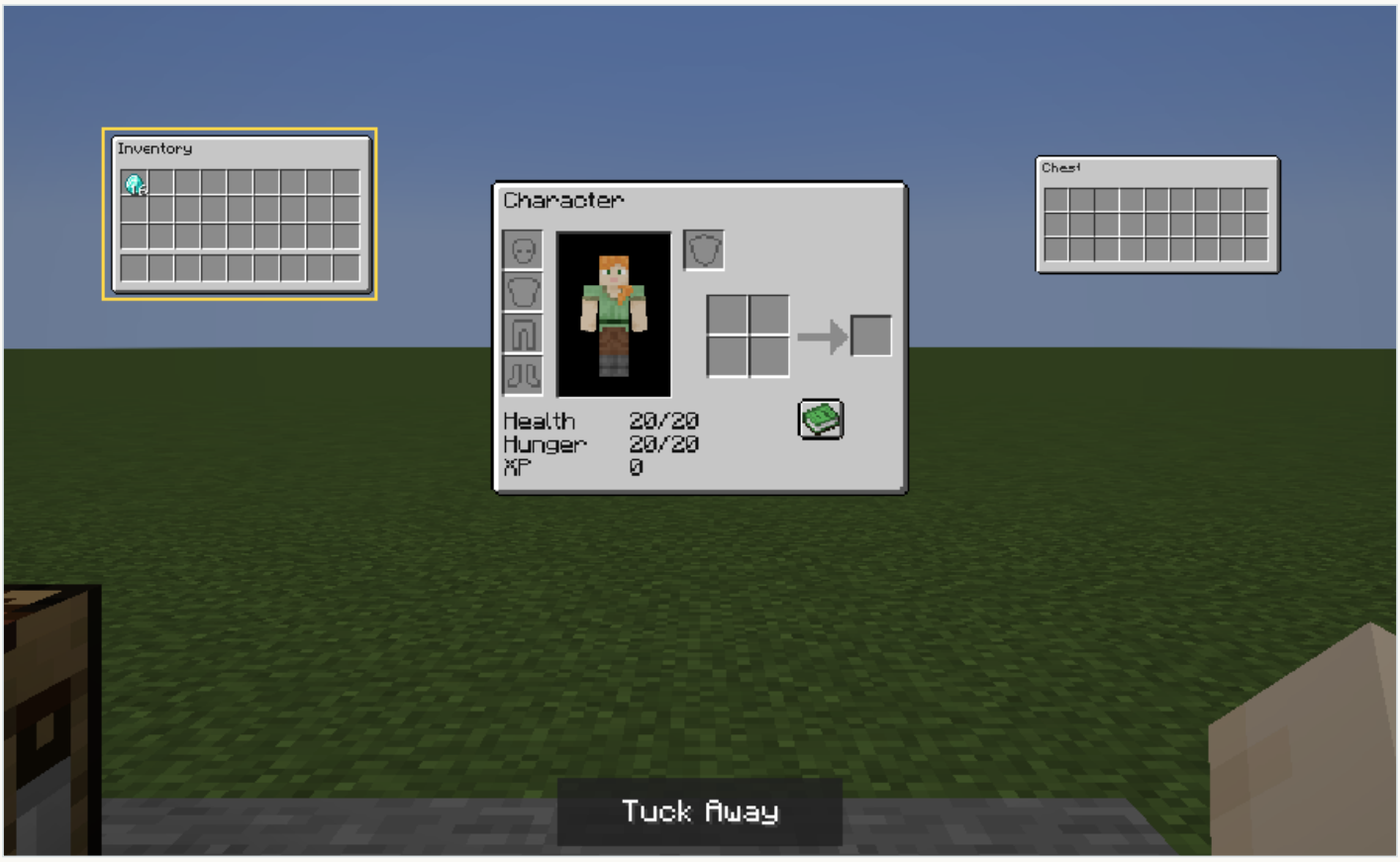
Controlify feature guide

A complete controller workspace for Salt's Inventory Update.

Minecraft 1.21.1 | Fabric + NeoForge | Version 0.1.4



IN GAME Character is the focused center window after a swap. Inventory remains available in the surrounding ring.

Three modes, with retained windows

Work in a large side-by-side pair, hold LT to choose a window in the radial, or tuck the entire workspace to resume gameplay.

This guide covers the new independent Controlify compatibility JAR and the generic Core API improvements it uses. Screenshots come from the final packaged Fabric verification run.

Branch: controllify-jar-test | Documented: 12 October 2026

Controls at a glance

Physical radial controls and the player's configured inventory actions.

SIDE BY SIDE



Interact with two large windows

RADIAL



Hold LT, point, then release

TUCKED



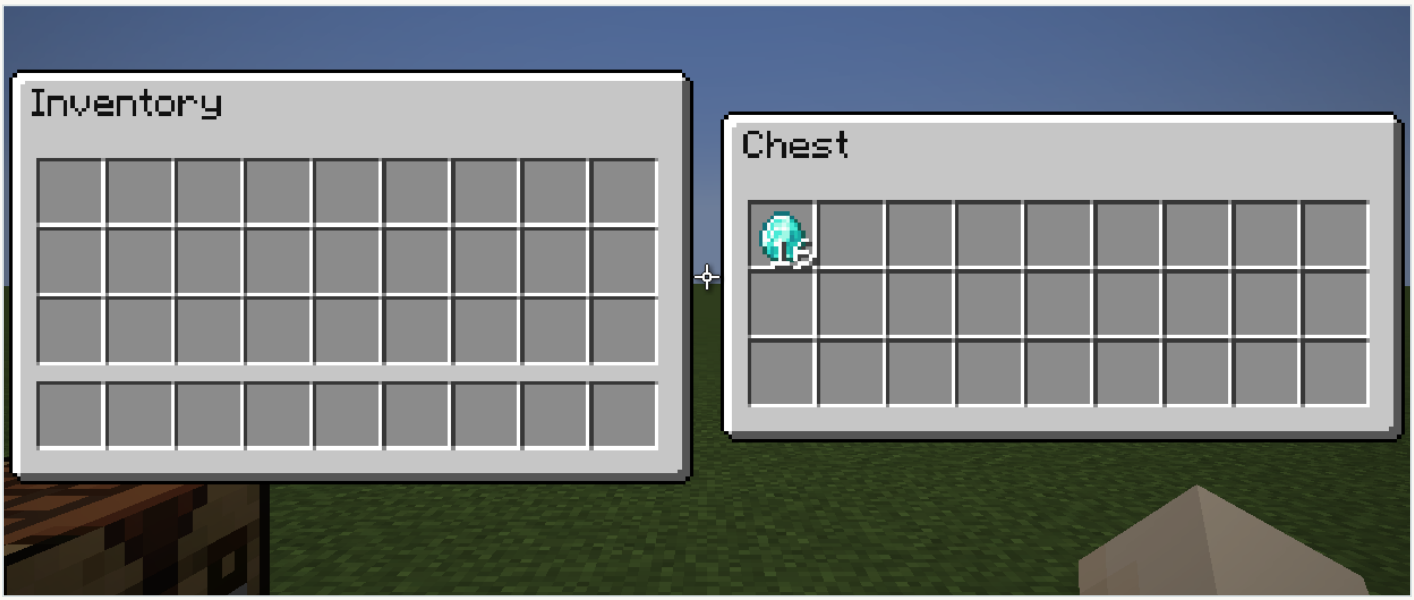
Normal gameplay; windows retained

Input	Action
LB / RB	Cycle the selected window counterclockwise / clockwise. Focus stays on the left.
Hold LT	Enter Radial Selection Mode.
Left stick + LT	Highlight the nearest radial window; the bottom sector selects Tuck Away.
Release LT	Select the highlight on the right, or commit Tuck Away.
A + LT	Swap the highlighted window into focus.
Close binding (normally B)	Short press/release while LT is held: close the highlighted closable window.
Hold Close	Close all closable windows in either visible mode, using Core's hold duration.
Inventory binding (normally Y)	Open Inventory from gameplay or restore the tucked workspace.
Native UI bindings	Controlify retains cursor, scrolling, select, quick-move and take-half behavior.

Quick-move uses its configured Controlify binding. RB is reserved for choosing the next window, not for transferring items.

Open a large, usable pair

Inventory is the default focused window; the opened container or UI is selected.



01 Opening a chest places Inventory on the left and Chest on the right.

Two visible windows

Only the focused and selected windows appear in Side-by-Side Mode. The focused window stays on the left; the selected window stays on the right. Other windows remain available through the radial.

Inventory and Character are permanent

Opening any supported container or UI also ensures Inventory is open. Inventory and Character cannot be closed individually, even after another window becomes focused.

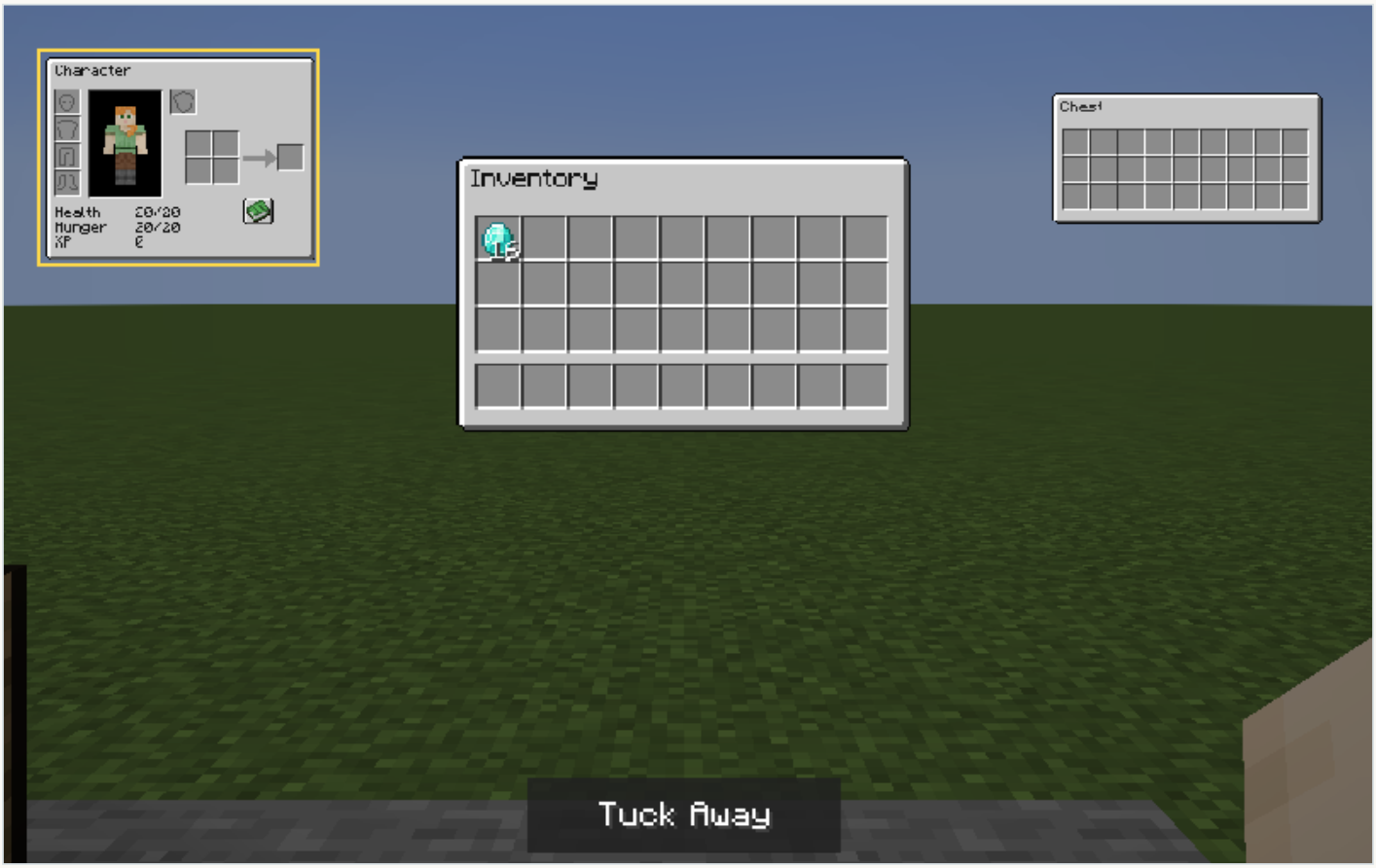
Switch the selected window without LT

RB chooses the next window clockwise; LB chooses the previous window counterclockwise. Cycling skips the focused center window and leaves the left side unchanged.

While either visible inventory mode is active, character movement and world interaction are blocked. Use Tuck Away to resume gameplay.

Choose a window in the radial

Hold LT to reveal the open windows around the focused center window.



02 The gold outline marks the highlighted window. The bottom sector is reserved for Tuck Away.

Hold, point, release

Holding LT shrinks the focused window and moves it to the center. Move the left stick toward another window to highlight the nearest radial position. Release LT to select it.

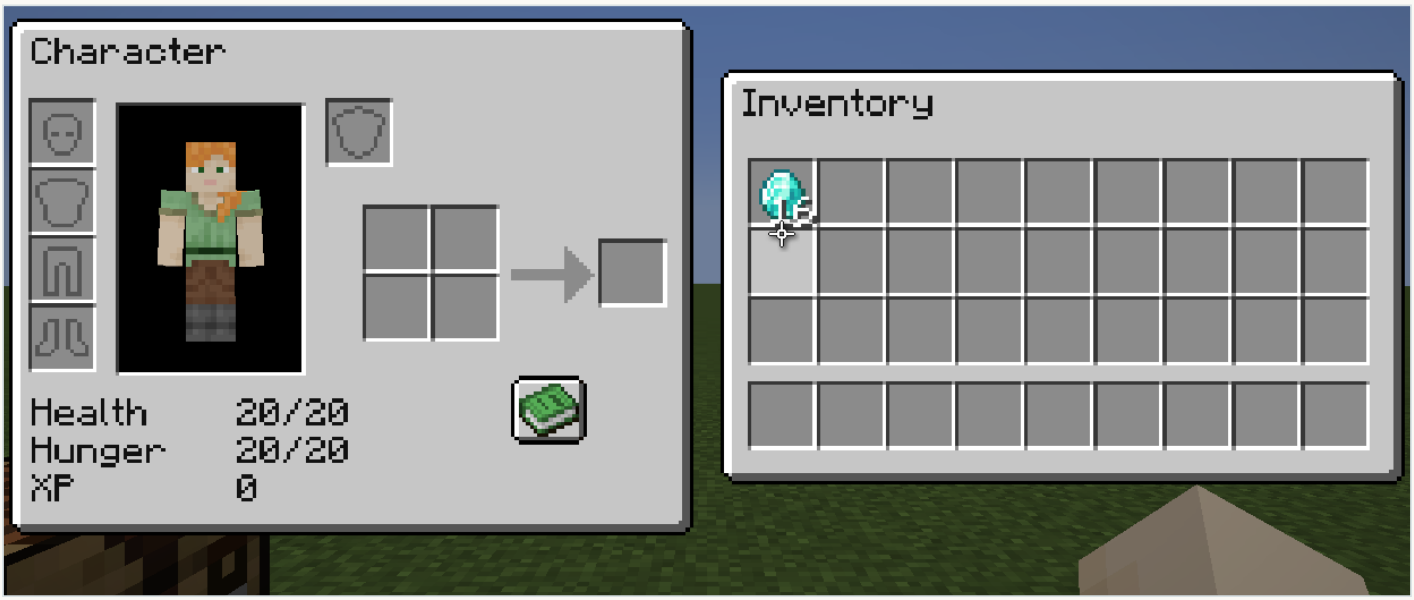
Return to the displayed pair

After release, the focused window returns to the left. The highlighted window becomes the large selected window on the right.

Radial selection owns the stick and action input. Controlify's virtual cursor position is retained for the return to normal UI interaction.

Make any window the focus

A swaps the highlighted window with the current focused window while LT is held.



04 After swapping and releasing LT, Character is focused on the left and Inventory is selected on the right.

Swap from the radial

Hold LT, highlight the desired window with the left stick, then press A. The highlighted window becomes the new center. The previously focused window moves into the surrounding ring.

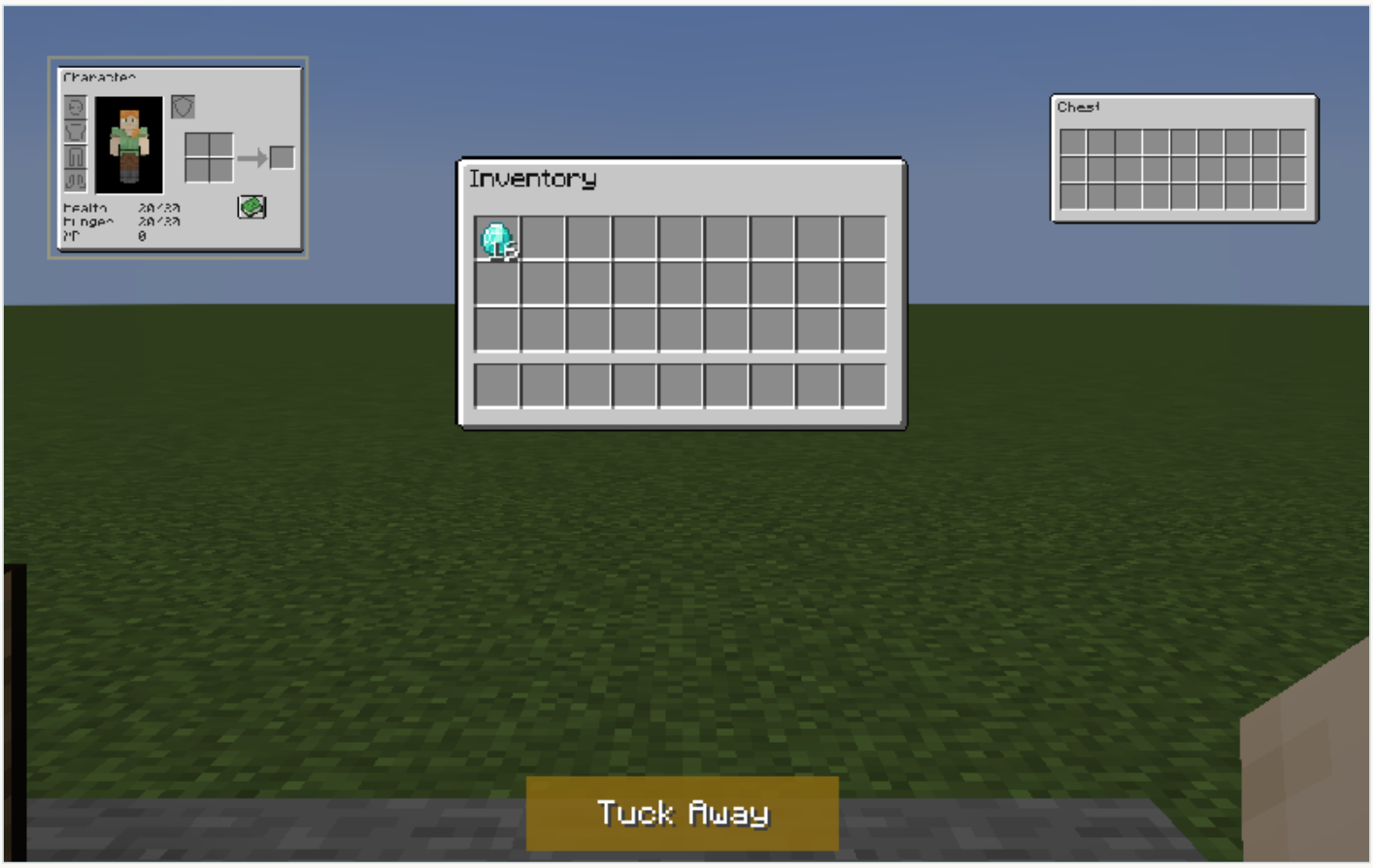
Focus is not tied to Inventory

A container, Character, or a supported UI can become focused. Inventory stays available even when it is no longer in the center.

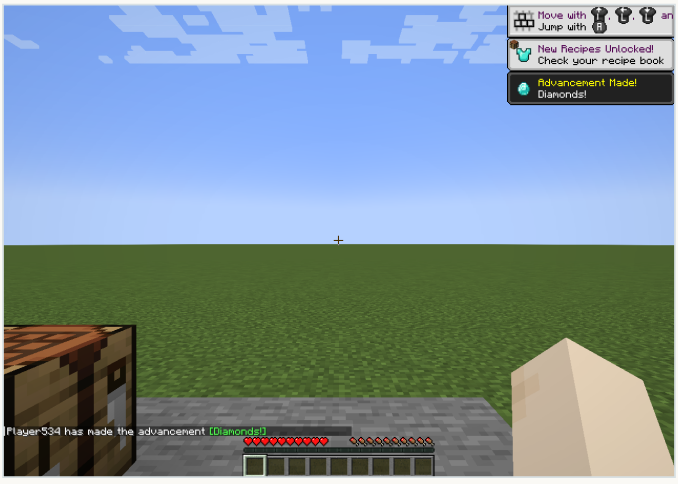
Releasing LT returns to Side-by-Side Mode with the new focused window on the left. Focus and selection remain separate roles.

Tuck the workspace and play

The whole workspace moves upward while every window remains open.



05 Point the left stick downward while holding LT, then release LT to commit Tuck Away.



06 Gameplay resumes; the windows are retained.

Full gameplay controls

Movement and normal world interaction resume as the screen returns to gameplay.

No closing or recreation

Tucking changes presentation only. Existing window models, UI state and authenticated container sessions stay alive.

Restore the same workspace

The configured inventory binding, normally Y, restores the tucked windows.



07 The restored visible pair is Inventory and Character. Other retained windows remain available through the radial.



13 A retained example UI keeps its click counters after tuck/restore and closing another view. This panel is a QA fixture.

Open more windows while tucked

Containers and supported UI windows opened during gameplay join the existing workspace without exposing the interface. They are available when you return.

The verification run checked unchanged window identities, model/state objects, container-session objects and authentication tokens across tuck and restore.

Quick-move to the other window

Transfers follow the displayed pair, independently of the desktop's old focus.



08 Before the tested transfer: Chest is focused and Barrel is selected. Quick-move targets Barrel.

A precise destination

Use Controlify's configured quick-move action over a stack in either displayed inventory. The other displayed inventory receives it. An unrelated open inventory is not used as a fallback.

Respect the target surface

Player storage and the Character surface are separate destinations. An inventory-less UI cannot accept a stack. A full or invalid exact destination rejects the transfer rather than using another window.

In the recorded scenario, 32 diamonds moved from Chest to Barrel while player storage stayed unchanged. Item conservation and authenticated server mutations were checked.

Keep native result behavior

Crafting and furnace output still use their native consumption and reward rules.



11a After the remaining crafting result transfers, the chestplate appears in Inventory and the crafting grid is empty.



11 Furnace output transferred into Inventory. The server also verified the earned experience.

Crafting results

Output transfers retain native recipe consumption and preserve a next result when ingredients remain. Transfers into player storage preserve vanilla repeated crafting. A full target leaves the source intact.

Furnace results

Native take bookkeeping receives the actual extracted count, and vanilla furnace XP is awarded. Unmoved output remains in the source; the shared carried stack is preserved.

Route repeated Loom outputs

Native result slot 3 supports exact transfers and recipe replenishment.



11b One patterned banner is in Barrel, while the next Loom result remains available. The native tooltip identifies the pattern.



11c After selecting Inventory, the remaining patterned banner transfers there and the Loom inputs/result are exhausted.

The recorded two-recipe transfer

The test used two banner/dye input pairs. The first transfer consumed one pair and replenished the output. The second transfer consumed the remaining pair without duplicating output or consuming extra inputs.

The same destination rules apply as for ordinary stacks. Native output handling also covers player crafting, anvils, cartography, grindstones, merchants, smithing and stonecutting.

Preserve native UI interactions

Controlify's cursor and the existing native controls remain the normal interaction path.



12b Controlify's normal click reaches the button outside the base rectangle. Both example counters are 1.



12c After an outside press, the native modal is dismissed exactly once. The cursor is above the owner window.

Native pointer behavior

Cursor movement, scrolling, selection, quick-move, take-half and D-pad slot snapping use Controlify's existing behavior.

Correct transformed geometry

External controls, popup regions and slot snapping follow the displayed scale and position. Native modal ownership and empty-area dismissal are retained.

The illustrated client-only panel is a test fixture, not an additional shipped interface. Its retained state and close callbacks demonstrate existing integration UI support.

Close windows safely

Individual closing and close-all are separate from tucking.



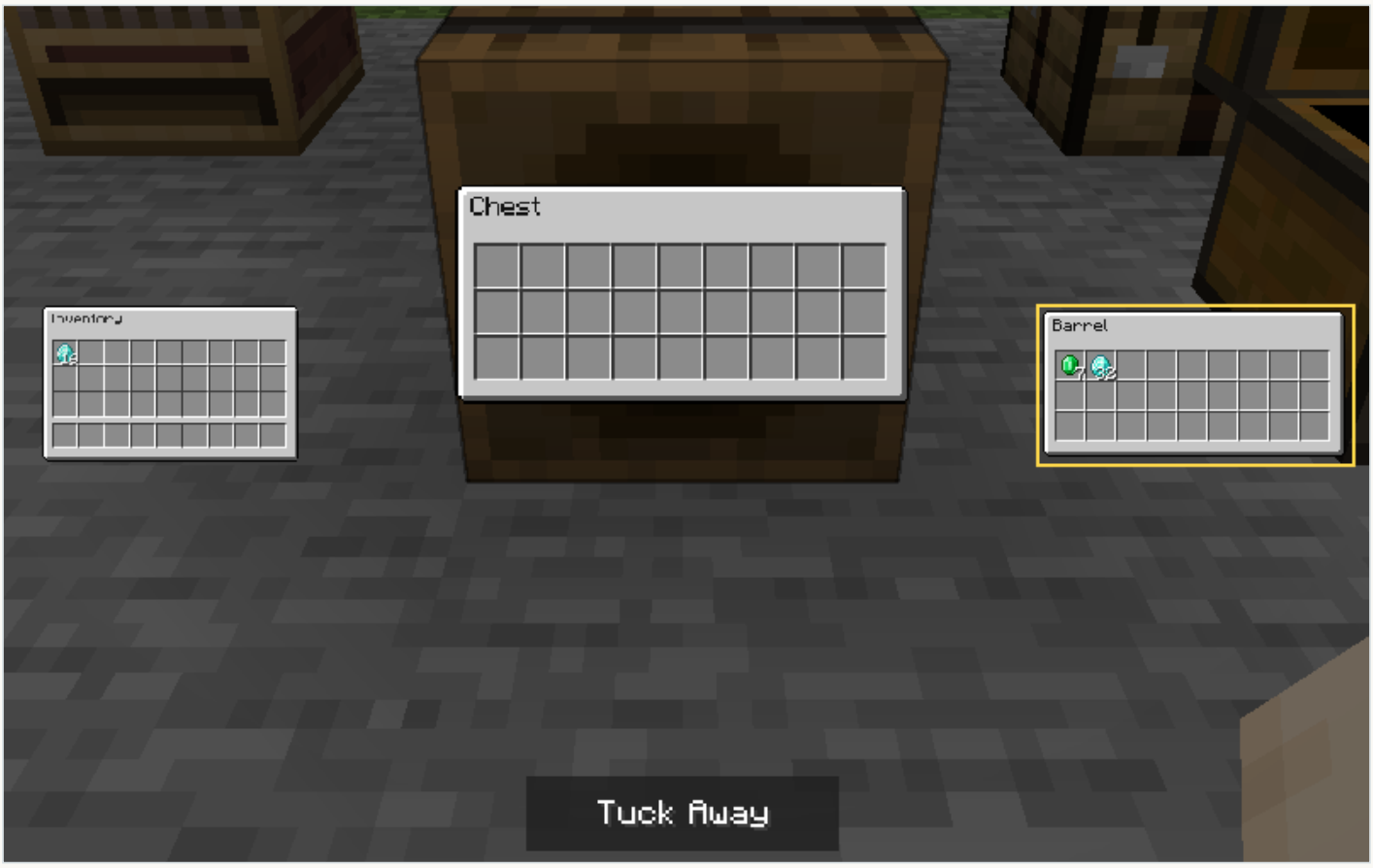
10 After holding the configured Close binding, closable windows have been closed and gameplay resumes.

Action	Result
Short Close + LT	On release, close the highlighted closable window. Inventory and Character stay available.
Hold Close	In Side-by-Side or Radial Mode, close all closable windows and return to gameplay.
Tuck Away	Hide the workspace and preserve every window and session.

Close is the player's configured close-inventory binding, normally B. The hold threshold comes from Core's existing hold-to-close-all setting. A short close tap in Side-by-Side Mode does not close a window.

Animations and window HUD

Smooth transitions by default, with an immediate-transition option.



09 A settled capture from the animations-disabled test. A still image does not demonstrate motion; immediate target placement was checked in the runtime.

Smooth movement and emphasis

Animations cover highlight changes, selection, center shrinking, pair/radial transitions, bumper cycling, focus swaps, tuck/restore and new windows.

Required inventory arrangement

The existing inventory HUD/hotbar and Character offhand arrangement is enabled for controller presentation without overwriting the saved keyboard/mouse preference.

Disable animations

Edit `config/salts_inventory_controlify.properties`

Set `animations=false` and restart the client. Controls and layouts continue working, with transitions applied immediately.

Install the separate companion

Builds are available for Minecraft 1.21.1 Fabric and NeoForge.

CORE

Inventory, sessions and public API

Required: rebuilt matching core

EXTENDED

Existing integrations for other mods

Optional

CONTROLLER JAR

Layout, input and animation

Required: third independent JAR

The companion contains controller layouts, input, animations and transformed clipping. It uses only Core's public API; Core retains the window models and container-session ownership.

Install the matching loader files

Use the rebuilt Core and Controlify companion from `dist/controlify-test/`. Install Controlify and its YACL dependency. Fabric also requires Fabric API and Fabric Loader 0.18.2 or later. Use Java 21.

> Older published Core 0.1.4 archives do not contain these workspace API hooks. Use the Core rebuilt from this branch with the companion. The pinned Controlify version is 3.0.1+lts.

Evidence	Final result
Automated Java tests	204 passed: 121 Core, 35 controller, 35 extended Fabric, 13 extended NeoForge.
Architecture isolation	Both controller compiler classpaths and all six Core/extended/companion JARs passed isolation checks.
Packaged Fabric run	Zero failures; 305 records: 286 PASS, 18 captures and 1 observation. Staged artifact hashes match the collected JARs.

Verification scope

The packaged gameplay run used the real Fabric game and an integrated server with a synthetic Controlify driver and real bindings. Physical controller hardware and packaged NeoForge gameplay were not tested. Other Minecraft versions are not built targets of this integration.

Sources: controlify/README.md; controlify/VALIDATION.md; final packaged QA report and screenshots in build/controlify-qa/1.21.1-fabric/attempt-19962-final-pass/. All screenshots are native captures from that 12 October 2026 run, cropped only for page layout.

Twelve native targets

Current compatibility, installation and runtime evidence

Pages 1-14 preserve the original illustrated feature guide and its Minecraft 1.21.1 Fabric captures. This appendix supersedes their installation matrix, test totals and verification-scope statements. The controls and feature descriptions remain applicable.

12/12 packaged runtimes verified. All twelve reports match the final collected Core and companion JARs.

Minecraft	Loader	Native controller	YACL	Runtime
1.20.1	Fabric	2.0.4	3.6.6+1.20.1-fabric	Verified
1.20.1	Forge	2.1.9 (Forge port)	3.6.6+1.20.1-forge	Verified
1.21.1	Fabric	3.0.1+lts	3.8.2+1.21.1-fabric	Verified
1.21.1	NeoForge	3.0.1+lts	3.8.2+1.21.1-neoforge	Verified
1.21.11	Fabric	3.0.1+lts	3.8.2+1.21.11-fabric	Verified
1.21.11	NeoForge	3.0.1+lts	3.8.2+1.21.11-neoforge	Verified
26.1.2	Fabric	3.5.3+mc26.1	3.9.7+26.1-fabric	Verified
26.1.2	NeoForge	3.5.3+mc26.1	3.9.7+26.1-neoforge	Verified
26.2	Fabric	3.5.3+mc26.2	3.9.7+26.2-fabric	Verified
26.2	NeoForge	3.5.3+mc26.2	3.9.7+26.2-neoforge	Verified
26.3	Fabric	3.5.3+mc26.3	3.9.7+26.3-fabric	Verified
26.3	NeoForge	3.5.3+mc26.3	3.9.7+26.3-neoforge	Verified

Controller and YACL labels link to the exact releases. 1.20.1 Forge also requires LexManos's Mixin Args 1.0 as a separate client mod. The 26.x upstream controller JARs have native Fabric/NeoForge entrypoints; each Salt companion is loader-specific.

Pin source: controlify/targets.json, including extra runtime dependencies and SHA-512 digests. Snapshot: 13 October 2026, 04:58 UTC.

Install matching native files

The third JAR and its matching rebuilt Core remain separate mods.

Choose one Minecraft version and loader

Install its rebuilt Salt Core and Controlify companion from dist/controlify-test/core/ and dist/controlify-test/controlify/. Their Salt version numbers must match. An older published Core archive with the same 0.1.4 label may lack these workspace hooks.

Add the exact native dependencies

Install the controller and YACL releases linked on page 15. Fabric also needs matching Fabric API. Forge uses Controlify: Forgified, Forge YACL and the separate original Mixin Args 1.0 JAR. NeoForge uses its native dependency builds. Extended compatibility remains optional.

Match Java and Fabric Loader

Java 17 for Minecraft 1.20.1; Java 21 for 1.21.1 and 1.21.11; Java 25 for every 26.x target. Fabric Loader 0.18.2 or newer for 1.20.1 and 1.21.1; 0.19.2 or newer for 1.21.11; 0.19.5 or newer for every 26.x target. The 26.x minimum provides the MixinExtras compatibility fix needed by the unchanged upstream mods.

Extra runtime dependency: 1.20.1 Forge: LexManos's Mixin Args 1.0. Install this unchanged upstream JAR separately; it is not bundled in the Salt companion.

Fabric Loader 0.19.5 release notes. The original controls and configuration on pages 1-14 apply across the matrix.

Existing 1.20.1 preview limit: Minecraft 1.20.1 retains Core's existing blank large Loom banner preview. Native pattern selection, result transfers, input consumption and replenishment work; the modern targets also render the banner preview.

What the current evidence establishes

The current seven JUnit report directories contain **768 tests** and 0 failures/errors. Core is tested once per Minecraft version; each loader additionally needs its own packaged runtime report. **12/12 packaged targets are verified** in this snapshot.

Runtime verification requires real controller bindings and screen processing, an integrated server, all required scenario assertions, native PNG captures, original pinned dependency hashes, and final Core/companion JAR hashes matching the collected installation bundle. The driver is synthetic; physical controller hardware coverage is not claimed.

Current sources: controlify/VALIDATION.md; tools/verify-controlify-runtime.py; each target's results.json and screenshots. Snapshot: 13 October 2026, 04:58 UTC.

Modern rendering: radial focus

Verified native capture: Minecraft 1.21.11, NeoForge



IN GAME Character is focused in the radial workspace. The surrounding previews use the native window renderers with the companion's transformed placement and queued opacity.

This target's completed run records **295 PASS assertions/steps** and **18 native screenshots**, with zero reported failures. The verifier confirms matching game/dependency versions, original dependency pins, and final staged/collected artifact hashes.

Capture: build/controlify-qa/1.21.11-neoforge/screenshots/03-swapped-focus-radial.png
Scenario evidence: character-now-focused
Report: build/controlify-qa/1.21.11-neoforge/results.json

Exact transfers between windows

Verified native capture: Minecraft 26.1.2, Fabric



IN GAME Two existing container sessions form the displayed pair. Quick-move routes to the other displayed storage window; the server checks destination authorization and item conservation.

This target's completed run records **293 PASS assertions/steps** and **18 native screenshots**, with zero reported failures. The verifier confirms matching game/dependency versions, original dependency pins, and final staged/collected artifact hashes.

Capture: build/controlify-qa/26.1.2-fabric/screenshots/08-two-containers-pair.png
Scenario evidence: server-exact-pair-and-conservation
Report: build/controlify-qa/26.1.2-fabric/results.json

Native UI beyond the base rectangle

Verified native capture: Minecraft 26.2, NeoForge



IN GAME The test view's external button receives the configured native A action through its own interactive region. Its window identity and UI state survive tuck and restoration.

This target's completed run records **299 PASS assertions/steps** and **18 native screenshots**, with zero reported failures. The verifier confirms matching game/dependency versions, original dependency pins, and final staged/collected artifact hashes.

Capture: build/controlify-qa/26.2-neoforge/screenshots/12b-external-client-UI-button.png
Scenario evidence: protruding-client-UI-button-clicked
Report: build/controlify-qa/26.2-neoforge/results.json

Native results and retained inputs

Verified native capture: Minecraft 26.3, Fabric

PACKAGED MODERN RUNTIME 20 / 20



IN GAME A Loom result moves into the selected container. Native callbacks consume one banner and dye, replenish the next result, and conserve the two completed banners across the displayed destinations.

This target's completed run records **292 PASS assertions/steps** and **18 native screenshots**, with zero reported failures. The verifier confirms matching game/dependency versions, original dependency pins, and final staged/collected artifact hashes.

Capture: build/controlify-qa/26.3-fabric/screenshots/11b-native-loom-replenished-result.png
Scenario evidence: loom-native-inputs-consumed-once-output-replenished
Report: build/controlify-qa/26.3-fabric/results.json